Task 1: Реализация многопоточной версии программы умножения матрицы на вектор

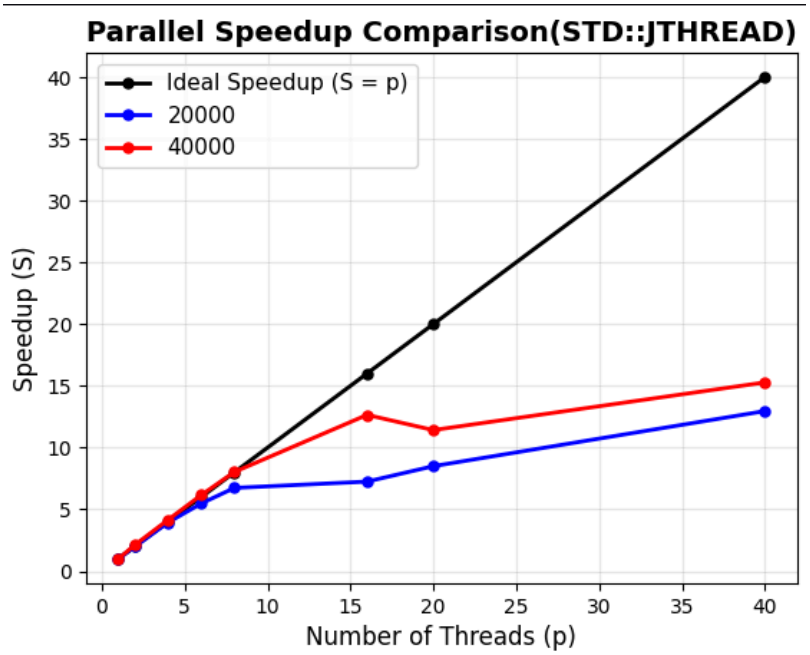
1. С помощью std::jthread:

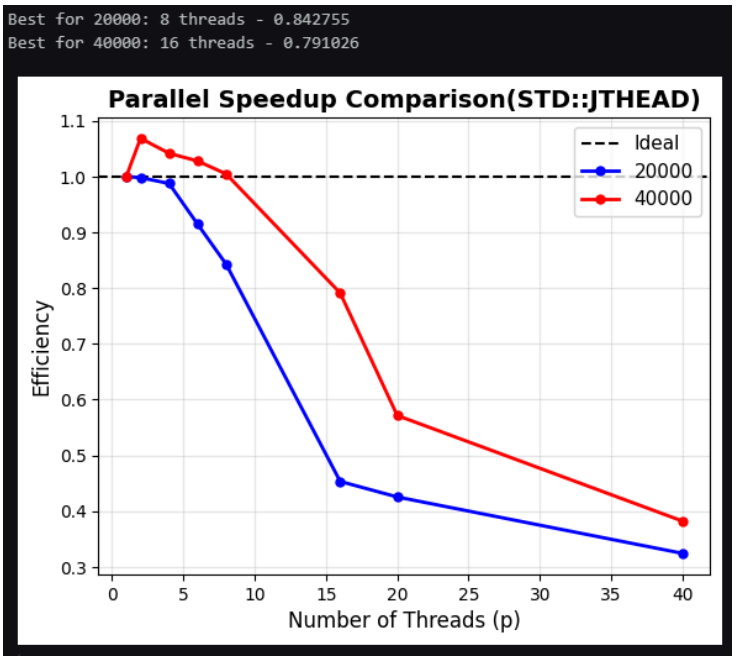
M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
20000	3.93	1	1.97	1.99	0.99	3.95	0.71	5.48	0.58	6.74	0.54	7.25	0.46	8.5	0.3	12.9
40000	16.0	1	7.92	2.13	4.06	4.16	2.74	6.16	2.1	8.03	1.33	12.6	1.48	11.4	1.1	15.3

Processor:

Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Architecture: x86_64
CPU(s): 80

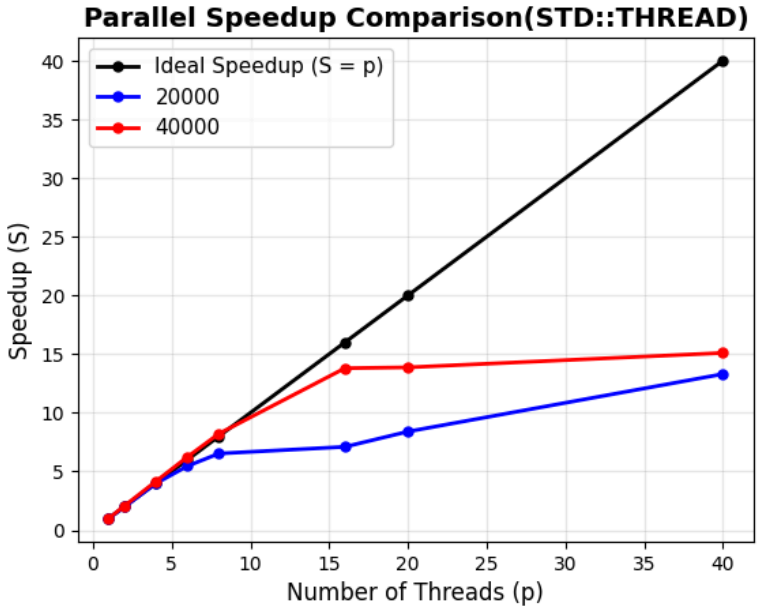
Server name: ProLiant XL270d Gen10
NUMA nodes count: Available: 2 nodes (0-1)
Node 0 size: 385636 MB
Node 1 size: 387008 MB
Operation System:
PRETTY_NAME = "Ubuntu 22.04.5 LTS"
VERSION = "22.04.5 LTS (Jammy Jellyfish)"

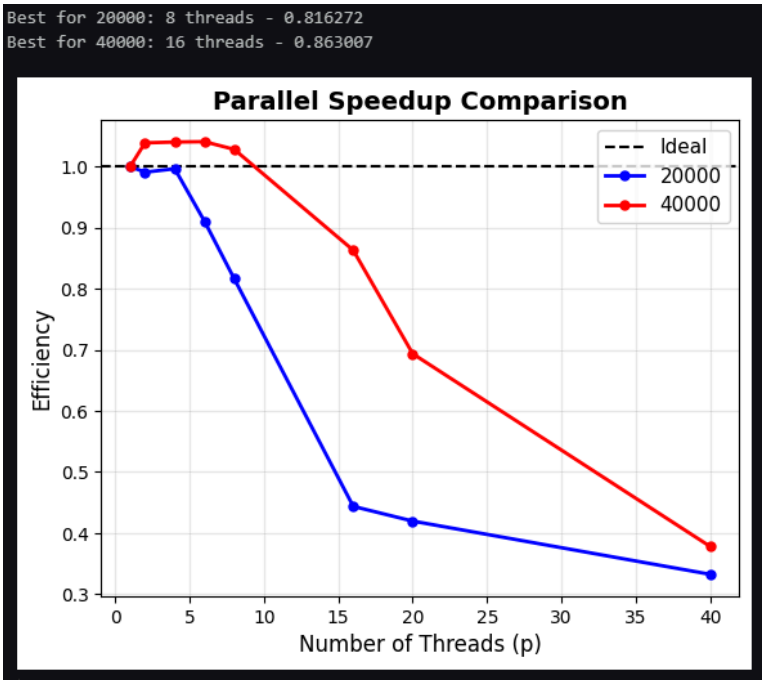




2. С помощью std::thread:

M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
20000	3.92	1	1.98	1.98	0.98	3.98	0.71	5.46	0.6	6.53	0.55	7.1	0.46	8.4	0.29	13.3
40000	16.9	1	8.16	2.07	4.07	4.16	2.71	6.24	2.06	8.22	1.22	13.8	1.22	13.8	1.12	15.1





Вывод:

На большом объеме данных разбиение задачи на параллельные сегменты с параллельной инициализацией дает огромный прирост скорости выполнения, парой выше, чем идеально ожидаемый прирост.